UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ

Portas de Entrada e Saída Botões e Display de 7 segmentos

Docente: Prof. Dr. Ruhan Pontes Policarpo de Souza
Discente: Carlos Gabriel Baratieri — 2706598
Discente: Luiz Fernando Coimbra Hintz — 2559145
Turma: C51B

O presente documento tem como objetivo relatar a segunda prática laboratorial da disciplina de Sistemas Microcontrolados, envolvendo portas de entrada e saída Botões e display de 7 segmentos.

Sistemas Microcontrolados — EC45C
15/09/2026

22 de setembro de 2026 — 14:12
Discente: Carlos Gabriel Baratieri
Discente: Luiz F. Coimbra Hintz

1 Introdução

O presente experimento tem como objetivo abordar o funcionamento geral do display de 7 segmentos por meio de ferramentas, como o Proteus e o compilador MPLAB X IDE, em conjunto com o kit didático XM118 – Exsto, fazendo uso de gravação interna. Foi disponibilizado previamente pelo professor o esquemático usado pelo Proteus para que fosse possível simular o firmware e, depois, gravá-lo no kit e testar o código em execução.

Sumariamente, o firmware era em um contador, com incremento de 5 e decremento de 2, com valor máximo de 4999, apresentado nos quatro displays de 7 segmentos disponíveis. A lógica do firmware será descrita ao longo do relatório. Cada display de 7 segmentos representava uma casa decimal, ou seja, unidade, dezena, centena e milhar.

2 Materiais e Métodos

Os materiais utilizados para a realização do experimento foram o microcontrolador PIC18F4550, o kit didático XM118 – Exsto (com PIC18F4550 e displays de 7 segmentos integrados), um computador com o MPLAB X IDE versão 5.20, o software Proteus para simulação e cabos para conexão. Na Figura 1, pode ser visto o kit didático utilizado no experimento, além dos displays de 7 segmentos em funcionamento.

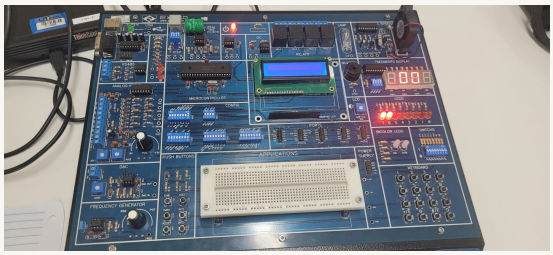


Figura 1: Kit didático XM118 com os displays de 7 segmentos em funcionamento.

Na Figura 2, pode ser visto o esquemático do Proteus disponibilizado previamente pelo professor.

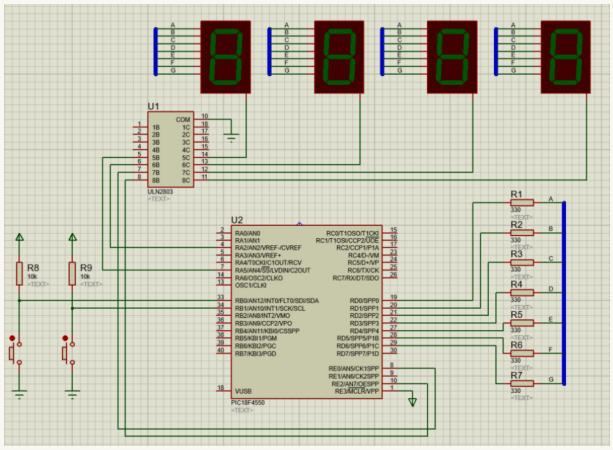


Figura 2: Esquemático do circuito no Proteus disponibilizado pelo professor.

Inicialmente, foi realizada a leitura do roteiro, junto com as explicações. Depois, abrimos o esquemático e o compilador. Como a lógica era simples, programamos e testamos primeiro no kit e no Proteus. Enunciado abaixo.

> Desenvolva um firmware capaz de apresentar o valor de uma variável com valor máximo de 4999 nos quatro displays de 7 segmentos disponíveis. Esse valor deve ser incrementado por RB0 e decrementado por RB1. O incremento deve ser de 5 unidades e o decremento deve ser de 2 unidades. O tempo de varredura dos displays deve garantir que a cintilação não seja desconfortável aos olhos dos usuários.

3 Resultados

Abaixo, temos o código que foi testado tanto no kit quanto no Proteus.

```c
#include <xc.h>

#pragma config PLLDIV = 5
#pragma config CPUDIV = OSC1_PLL2
#pragma config FOSC = HS
#pragma config PBADEN = OFF
#pragma config LVP = OFF
#pragma config DEBUG = ON

#define _XTAL_FREQ 20000000

void delay(void) {
    __delay_ms(200);
}

void main(void) {
    unsigned char disp7s[] = {0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07, 0x7F, 0x6F};
    unsigned int valor = 0;
    unsigned int milhar, centena, dezena, unidade;
    TRISA = 0;
    TRISD = 0;
    TRISE = 0;
    TRISB = 0b00000011;

    while (1) {
        if (PORTBbits.RB0 == 0) {
            if (valor <= 4997) {
                valor += 5;
            } else {
                valor = 4999;
            }
            // delay();
        }

        if (PORTBbits.RB1 == 0) {
            if (valor >= 2) {
                valor -= 2;
            } else {
                valor = 0;
            }
            // delay();
        }

        milhar = valor / 1000;
        centena = (valor / 100) % 10;
        dezena = (valor / 10) % 10;
        unidade = valor % 10;

        LATEbits.LATE2 = 1;
        LATEbits.LATE0 = 0;
        LATAbits.LATA2 = 0;
        LATAbits.LATA5 = 0;

        LATD = disp7s[unidade];
        delay();

        LATEbits.LATE2 = 0;
        LATEbits.LATE0 = 1;
        LATAbits.LATA2 = 0;
        LATAbits.LATA5 = 0;

        LATD = disp7s[dezena];
        delay();

        LATEbits.LATE2 = 0;
        LATEbits.LATE0 = 0;
        LATAbits.LATA2 = 1;
        LATAbits.LATA5 = 0;

        LATD = disp7s[centena];
        delay();

        LATEbits.LATE2 = 0;
        LATEbits.LATE0 = 0;
        LATAbits.LATA2 = 0;
        LATAbits.LATA5 = 1;

        LATD = disp7s[milhar];
        delay();
    }
}
```



Primeiramente, são definidas a biblioteca e as configurações iniciais do microcontrolador. Essas partes foram baseadas no modelo disponibilizado pelo professor no PDF. É utilizada a biblioteca do XC8, algumas configurações internas do próprio PIC e, por fim, o macro que define a frequência do clock do microcontrolador.

```
// Biblioteca XC8
#include <xc.h>

// Configurações internas
#pragma config PLLDIV = 5
#pragma config CPUDIV = OSC1_PLL2
#pragma config FOSC = HS
#pragma config PBADEN = OFF
#pragma config LVP = OFF
#pragma config DEBUG = ON

// Macro
#define _XTAL_FREQ 20000000
```

Em seguida, foi criada uma função para controlar o tempo de atraso, de acordo com as condições estabelecidas no enunciado.

```
void delay(void) {
    __delay_ms(200);
}
```

O professor também disponibilizou alguns trechos de código, entre eles o vetor utilizado para controlar os displays de 7 segmentos. Esse vetor possui uma função importante, pois fornece os valores correspondentes aos números que serão apresentados nos displays. Os valores estão representados em hexadecimal, o que permite uma representação mais compacta dos códigos utilizados para acionar os segmentos. Cada posição do vetor corresponde a um dos algarismos de 0 a 9. Assim, caso queiramos acessar a primeira posição `disp7s[0]`, teríamos um valor de 0 representado no display.

```
void main(void) {
    unsigned char disp7s[] = {0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07, 0x7F, 0x6F};
    // [...]
}
```

Depois, foram definidas algumas variáveis que serão explicadas ao decorrer do código, mas elas são importantes para a lógica.

```
void main(void) {
    unsigned char disp7s[] = {0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07, 0x7F, 0x6F};
    unsigned int valor = 0;
    unsigned int milhar, centena, dezena, unidade;
    // [...]
}
```

Depois, foram configuradas as entradas e saídas utilizando os registradores `TRIS`, nos quais 1 representa entrada e 0 representa saída. O `TRISA` configura os pinos da PORTA como saída, assim como o `TRISD` configura os pinos da PORTD e o `TRISE` configura os pinos da PORTE. Já o `TRISB` foi configurado para que os pinos RB0 e RB1 fossem utilizados como entradas, responsáveis pela leitura dos botões.

```
void main(void) {
    unsigned char disp7s[] = {0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07, 0x7F, 0x6F};
    unsigned int valor = 0;
    unsigned int milhar, centena, dezena, unidade;
    TRISA = 0;
    TRISD = 0;
    TRISE = 0;
    TRISB = 0b00000011;
    // [...]
}
```

Apenas para ilustrar, a configuração das portas por meio do registrador funciona da seguinte forma:

```
RB7 RB6 RB5 RB4 RB3 RB2 RB1 RB0
 0   0   0   0   0   0   1   1
```

Como dito anteriormente, o valor 1 representa uma entrada, o que permite configurar os botões mencionados no enunciado. O mesmo funcionamento se aplica às demais portas.

Para as outras, foi atribuído um valor decimal, que será convertido para binário. É como se estivéssemos escrevendo diretamente um valor no formato `0b00000000`.



Logo após, realizamos um looping infinito, para que o microcontrolador fique verificando os comandos.

```
void main(void) {
    unsigned char disp7s[] = {0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07, 0x7F, 0x6F};
    unsigned int valor = 0;
    unsigned int milhar, centena, dezena, unidade;
    TRISA = 0;
    TRISD = 0;
    TRISE = 0;
    TRISB = 0b00000011;

    while (1) {
        // [...]
    }
}
```

Depois, foi escrita a lógica de acionamento dos botões.

```
void main(void) {
    unsigned char disp7s[] = {0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07, 0x7F, 0x6F};
    unsigned int valor = 0;
    unsigned int milhar, centena, dezena, unidade;
    TRISA = 0;
    TRISD = 0;
    TRISE = 0;
    TRISB = 0b00000011;

    while (1) {
        if (PORTBbits.RB0 == 0) {
            // [...]
        }

        if (PORTBbits.RB1 == 0) {
            // [...]
        }
    }
}
```

O enunciado estabelece que o botão RB0 deve realizar o incremento de 5 unidades, enquanto o botão RB1 deve realizar o decremento de 2 unidades. Além disso, o valor máximo que pode ser apresentado nos displays é 4999. Dessa forma, foi escrita a seguinte parte da lógica:

```
void main(void) {
    unsigned char disp7s[] = {0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07, 0x7F, 0x6F};
    unsigned int valor = 0;
    unsigned int milhar, centena, dezena, unidade;
    TRISA = 0;
    TRISD = 0;
    TRISE = 0;
    TRISB = 0b00000011;

    while (1) {
        if (PORTBbits.RB0 == 0) {
            if (valor <= 4997) {
                valor += 5;
            } else {
                valor = 4999;
            }
            // delay();
        }

        if (PORTBbits.RB1 == 0) {
            if (valor >= 2) {
                valor -= 2;
            } else {
                valor = 0;
            }
            // delay();
        }
    }
}
```

A instrução `valor += 5` realiza o incremento de 5 unidades, enquanto `valor -= 2` realiza o decremento de 2 unidades. Foi usado condicionais para controlar os limites.

No caso do incremento, foi utilizada a condição `valor <= 4997`. Isso ocorre porque o limite máximo de apresentação é 4999. Assim, quando o valor for 4997, o incremento de 5 unidades resultaria em 5002, ultrapassando o limite estabelecido. Por esse motivo, quando o valor estiver acima de 4997, a condição `else` atribui diretamente o valor 4999, impedindo que o valor apresentado ultrapasse o limite máximo. No decremento, foi utilizada a condição `valor >= 2`, pois, caso o valor fosse 1, o decremento de 2 unidades resultaria em -1. Dessa forma, o valor é diretamente atribuído a 0.



No código mesmo, testado em sala de aula, foram usadas várias funções de `delay()`. Usamos e controlamos elas para ver como seria a velocidade, pois, como foi explicado em sala de aula, tanto os botões quanto os displays precisam ter esse controle. Logo, em código real, usamos 4 funções para controlar bem os botões, mas isso variou se era no Proteus ou se era no kit.

```
void main(void) {
    unsigned char disp7s[] = {0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07, 0x7F, 0x6F};
    unsigned int valor = 0;
    unsigned int milhar, centena, dezena, unidade;
    TRISA = 0;
    TRISD = 0;
    TRISE = 0;
    TRISB = 0b00000011;

    while (1) {
        if (PORTBbits.RB0 == 0) {
            if (valor <= 4997) {
                valor += 5;
            } else {
                valor = 4999;
            }
            delay();
            delay();
            delay();
            delay();
        }

        if (PORTBbits.RB1 == 0) {
            if (valor >= 2) {
                valor -= 2;
            } else {
                valor = 0;
            }
            delay();
            delay();
            delay();
            delay();
        }
    }
}
```

Agora vem a parte principal da lógica do código. Foi pensado o seguinte: temos 4 displays, logo, separamos isso em milhar, centena, dezena e unidade, como foi estabelecido nas variáveis iniciais. O valor inicial dos displays é 0. Assim, vamos tentar fazer um teste de mesa para explicar a lógica do código abaixo.

```
    milhar = valor / 1000;
    centena = (valor / 100) % 10;
    dezena = (valor / 10) % 10;
    unidade = valor % 10;
```

Vamos começar com o valor inicial, sendo 0, ou seja:

```
    milhar = valor / 1000; // Retorna: 0
    centena = (valor / 100) % 10; // Retorna: 0
    dezena = (valor / 10) % 10; // Retorna: 0
    unidade = valor % 10; // Retorna: 0
```

Se a gente incrementa 5, ficamos apenas com a unidade tendo valor:

```
    milhar = valor / 1000; // Retorna: 0
    centena = (valor / 100) % 10; // Retorna: 0
    dezena = (valor / 10) % 10; // Retorna: 0
    unidade = valor % 10; // Retorna: 5
```

No decremento de 2, ficamos com valor 3, ficamos apenas com a unidade tendo valor:

```
    milhar = valor / 1000; // Retorna: 0
    centena = (valor / 100) % 10; // Retorna: 0
    dezena = (valor / 10) % 10; // Retorna: 0
    unidade = valor % 10; // Retorna: 3
```

Vamos supor que estejamos em 1234 de valor, assim temos todas as casas com valor:

```
    milhar = valor / 1000; // Retorna: 1
    centena = (valor / 100) % 10; // Retorna: 2
    dezena = (valor / 10) % 10; // Retorna: 3
    unidade = valor % 10; // Retorna: 4
```

Ou seja, de forma resumida, essas expressões matemáticas separam os números das casas e os guardam em variáveis. Assim, colocamos esse retorno em cada um dos displays.

Explicando um pouco as expressões, vamos pegar como exemplo o valor 1234.

```
milhar = valor / 1000;
```

Estamos fazendo uma operação de divisão. Porém, como estamos lidando com números com tipagem de inteiros, ele apenas vai retornar o número inteiro, descartando a parte decimal. Matematicamente, daria o valor 1,234, mas, como descartamos a parte decimal, ficamos apenas com 1.

```
centena = (valor / 100) % 10;
```

Temos que primeiro realizar a divisão de 1234 por 100, que resulta em 12, pois a parte decimal é descartada. Depois, utilizamos o operador %, que retorna o resto de uma divisão. Nesse caso, temos 12 % 10, cujo resto é 2.

```
dezena = (valor / 10) % 10;
```

Primeiro realizamos a divisão de 1234 por 10, obtendo 123. Em seguida, utilizamos o operador % 10, que calcula o resto da divisão de 123 por 10. O resultado é 3.

```
unidade = valor % 10;
```

Nesse caso, utilizamos diretamente o operador % para obter o resto da divisão de 1234 por 10. Resultando 4.
Por fim, jogamos esses valores nos displays.

```
        LATEbits.LATE2 = 1;
        LATEbits.LATE0 = 0;
        LATAbits.LATA2 = 0;
        LATAbits.LATA5 = 0;

        LATD = disp7s[unidade];
        delay();

        LATEbits.LATE2 = 0;
        LATEbits.LATE0 = 1;
        LATAbits.LATA2 = 0;
        LATAbits.LATA5 = 0;

        LATD = disp7s[dezena];
        delay();

        LATEbits.LATE2 = 0;
        LATEbits.LATE0 = 0;
        LATAbits.LATA2 = 1;
        LATAbits.LATA5 = 0;

        LATD = disp7s[centena];
        delay();

        LATEbits.LATE2 = 0;
        LATEbits.LATE0 = 0;
        LATAbits.LATA2 = 0;
        LATAbits.LATA5 = 1;

        LATD = disp7s[milhar];
        delay();
```

Os displays compartilham o mesmo barramento, assim, é preciso acionar um display por vez, ligando a coluna do display a ser utilizado e desligando as demais, utilizando os pinos `LATE2`, `LATE0`, `LATA2` e `LATA5`.

Para cada display, apenas o pino correspondente é colocado em nível alto (1), enquanto os outros permanecem em nível baixo (0). Ao mesmo tempo, o valor correspondente ao dígito é enviado para a `LATD`, utilizando o vetor `disp7s`.

```
LATEbits.LATE2 = 1;
LATEbits.LATE0 = 0;
LATAbits.LATA2 = 0;
LATAbits.LATA5 = 0;

LATD = disp7s[unidade];
delay();
```

Nesse caso, `LATE2` recebe 1, enquanto os demais pinos recebem 0, ativando o display correspondente à unidade. Em seguida, `LATD` recebe `disp7s[unidade]`, fazendo com que o valor armazenado em `unidade` seja apresentado no display. No exemplo do 1234, ficaria `disp7s[4]`, utilizando a posição 4 do vetor, que corresponde ao valor 4. A função `delay()` é usada para controlar o tempo de apresentação.

Ao revezarmos os displays em alta velocidade, criamos a ilusão de que estão todos acesos, mas é possível verificar, com uma câmera digital, que, na verdade, na maior parte do tempo, todos os displays estão desligados.

4 Conclusão

Portanto, neste experimento, foi abordada a implementação de um firmware para o microcontrolador PIC18F4550, com foco nos displays de 7 segmentos. Foi realizada a simulação do firmware por meio do Proteus e, posteriormente, um teste prático no kit didático XM118 – Exsto. Além disso, o firmware foi compilado utilizando o MPLAB X IDE 5.20.

O firmware apresentou bom funcionamento, realizando os incrementos e decrementos nos displays indicados, respeitando os limites impostos pelo professor e as casas de unidade, dezena, centena e milhar em cada um dos displays, apresentando uma boa distribuição. Não ocorreu nenhum defeito no kit, e tudo funcionou de primeira. Dessa forma, os objetivos propostos para o experimento foram alcançados.